Budget Guard: Calibrated Resource Control for Long-Horizon LLM Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

An agent partway through a long task cannot say how much more it needs. Asked mid-task for an interval on its remaining tokens, Claude Opus 4.7 contains the truth 51%, 23%, and 35% of the time on Sokoban, Search-R1, and SWE-bench, against a 90% target. Budget Guard supplies that number from outside the agent. It reads the per-turn usage counts every provider API already reports, fits a small model to them, and wraps that model in conformal calibration, so that its interval on the remaining requirement contains the truth as often as promised. Nothing in the agent is trained or changed. Budget awareness turns out to be two questions, how much more and whether to stop, and we build a separately calibrated instrument for each. The intervals reach 0.90–0.96 coverage offline and 0.87–0.92 online with no unbounded intervals, hold at least 0.85 in 18 of 19 model–environment runs on one split each, and are sharper than a feature-free baseline; that much is guaranteed in distribution, and the experiments map what happens outside it. A guard moved from Sokoban to SWE-bench without recalibration falls to 0.10 coverage; recalibrating from finished runs recovers 0.80, and no bounded method reaches the target after a model swap. When three budgets bind at once, per-resource calibration covers the joint event 80% of the time; a Bonferroni correction restores 0.95 in distribution. The stopping instrument, a feasibility gate judged on money in Warehouse replay, avoids 86.3% of the spend on runs that could not finish, stops no run that would have finished, and cuts cost per success from \$1.61M to \$0.87M in simulated dollars, which no hard cap approaches. Feeding the intervals into the stop rule only makes it more conservative: the intervals measure, the gate decides. All stopping results are counterfactual replays of logged runs.

1 Introduction

LLM agents now take on tasks that run for many steps, many tool calls, and many interactions with an environment [1], and the resources they consume grow with them [2]. A single attempt can burn thousands of tokens, real time, and real money before it ends [3], much of it on reasoning that changes nothing [4–6]; when the attempt fails, nearly all of it was wasted [3]. Keeping that spending under control without giving up task performance is one of the practical obstacles to running agents at scale [7–9].

The obvious fix is to ask the agent, and it does not know. BAGEN [13] asked agents mid-task for an interval on how many more tokens they would need. For Claude Opus 4.7 those intervals contain the true number 51%, 23%, and 35% of the time on Sokoban, Search-R1, and SWE-bench, against a 90% target. Figure 1 shows what that looks like on one run: the agent's interval looks precise and sits above the truth from the second turn on, reserving 400 to 900 tokens for a run that needed 267.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

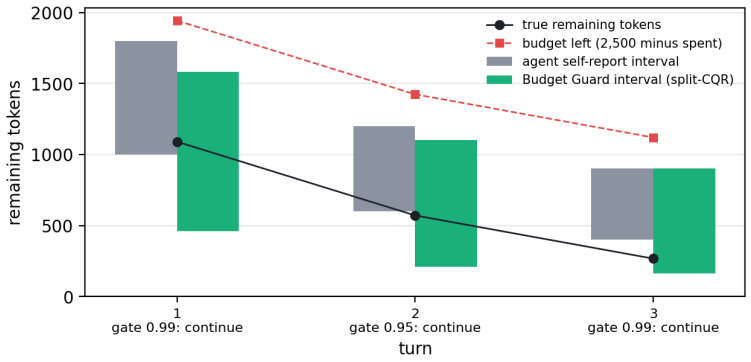


Figure 1: What Budget Guard does on one real run: the first held-out Sokoban rollout in our seed-0 test stream (Claude Opus 4.7, 2,500-token budget), chosen by that rule rather than by hand. Grey bars are the agent's own interval on the tokens it still needs, green bars the guard's, black dots the truth, and the dashed line the budget still available. The agent's interval misses from turn 2; the guard's contains the truth at every turn and stays under the remaining budget, and the guard's separate stop-or-continue gate (scores under each turn) says continue throughout. The run succeeded inside budget. Table 6 in Appendix B gives the numbers.

Budget Guard supplies the missing number from outside the agent. It is a wrapper: it reads the per-turn usage that provider APIs already report, fits a small gradient-boosted model to it, and wraps that model in a conformal calibration layer, so that its interval on the remaining requirement contains the truth as often as promised. Nothing in the agent is trained or changed, and because the guard reads only usage counts it works for closed models whose internals are unavailable. In Figure 1 the green intervals are wider than the agent's and right every time; spend so far plus their upper bound stays under the budget, so a deployer could have certified from the first turn that the run fit. That is the first half of budget awareness: how much more.

The second half is whether to stop, and it is a different question with a different instrument. A feasibility gate on the same features scores each turn and aborts a run when the score drops below a threshold calibrated so that at most a chosen fraction δ of runs that would have finished are stopped. We keep the two apart throughout, with separate calibration targets, separate metrics, and separate claims, because the results say we must: the intervals are a reliable measurement, and using them as a stopping signal only makes the gate more timid.

Two facts about the label shape everything. The resources a run still needs are known only when it ends, so online methods may read a label only at rollout boundaries; an earlier revision of this evaluation read them per prefix, and correcting that lowered our headline cross-environment recovery from 0.90 to 0.80. And the label does not exist at all for a run that can no longer finish, which is exactly the run a guard exists to catch. Half of our Warehouse prefixes are in that state.

Our findings, in order. (1) In distribution the intervals do what the construction promises, 0.90–0.96 coverage offline and 0.87–0.92 online, with no unbounded intervals and sharper bounds than a feature-free baseline on two of three environments; the agent's own estimate adds nothing as a feature (§5.1). (2) A frozen guard collapses under an environment change, from 0.92 to 0.10, and rollout-safe recalibration recovers 0.71 and 0.80 with bounded intervals; the methods that recover more do so with unbounded intervals on 60–88% of the stream, and nothing bounded reaches the target after a model swap (§5.2). (3) When several budgets bind at once, per-resource calibration under-covers the joint event, and a Bonferroni correction restores it in distribution but not under swap (§5.3). (4) The gate avoids 86.3% of doomed-run spend in Warehouse replay with no false aborts, which no hard cap achieves; on token tasks it saves far less, and adding the intervals to the rule makes it worse (§5.4). (5) Runs that cannot finish carry information that dropping them throws away (§5.5). Table 4 says which of these are guarantees and which are observations, and every number regenerates from one command over a pinned dataset.

2 Related work

BAGEN [13] defines budget awareness as an agent's ability to estimate mid-execution what it still needs, and asks the agent directly for an interval on remaining tokens and a judgment of whether finishing is still feasible, over internal resources such as tokens and external ones such as time, money, and warehouse capacity, on Sokoban, Search-R1, SWE-bench, and Warehouse. Its central finding is a gap between capability and self-knowledge: strong agents solve hard tasks while remaining poorly calibrated about what finishing will cost. Stopping early on the agent's own signals saves roughly 28–64% of tokens on failed trajectories at a cost of roughly 1.6–4.2 percentage points of task success, and interval coverage reaches only 47% even after supervised fine-tuning and reinforcement learning [13]. The agent carries some signal about its future; its raw estimate is not reliable enough to act on. Conformal prediction [14, 15] is the standard tool for turning an unreliable predictor into an interval with a stated coverage rate, and that is what we use it for.

Doomed From the Start [16] reads a model's internal activations to decide that a trajectory is doomed and stops it early. Like us, and unlike BAGEN, it uses a distribution-free calibrated gate with no fine-tuning, meeting recall targets of 90–97% on TextCraft and saving 47.1% (Qwen-2.5-7B) and 37.2% (Llama-3.2-3B) of inference compute at the 90% target. Because it needs internal states it applies only to open-weight models; Budget Guard needs only usage counts. MiCP [17] applies conformal prediction to a different object, the agent's answer, allocating an error budget across the turns of a multi-turn reasoning process so the agent can stop once it has enough evidence. We calibrate coverage over the remaining resource instead, sit outside the reasoning loop, and keep the abort gate separate from the intervals. Section 3 specifies the construction; Appendix A defines the terms used to read the results.

3 Method

The interval. Take any predictor, measure how badly it missed on data it never trained on, and widen its output by a high quantile of those misses. We fit two quantile regressors, at $\alpha/2$ and $1-\alpha/2$, giving a band $[\hat{f}_{\mathrm{lo}}(x), \hat{f}_{\mathrm{hi}}(x)]$; on each of n calibration points we record how far the truth fell outside it, $s = \max(\hat{f}_{\mathrm{lo}} - y,\ y - \hat{f}_{\mathrm{hi}})$, negative when the point sat inside; and with $\hat{q}$ the $\lceil (n+1)(1-\alpha) \rceil / n$ empirical quantile of those scores, the interval $[\hat{f}_{\mathrm{lo}}(x) - \hat{q},\ \hat{f}_{\mathrm{hi}}(x) + \hat{q}]$ contains y with probability at least $1-\alpha$ [10]. We use α=0.10 and clip the lower endpoint at zero. Two things about this guarantee matter for everything that follows. It is a hit rate over runs, not a property of any one interval: across exchangeable runs the interval contains the truth at least 90% of the time, and it says nothing about which 10% it misses. And it asks nothing of the data except that the calibration runs and the deployment runs be exchangeable. Everything difficult in this paper comes from the fact that they often are not. Table 4 records, for each component, exactly what we claim and under what assumption.

What the guard sees. At turn k the guard sees only the past (Figure 2): how far along the run is (the turn index); how much it has consumed (spend so far, and spend per turn, the burn rate); how it has been consuming recently (the last turn, the mean of the last three, and the mean, standard deviation, minimum, maximum, and trend of per-turn spend); and the agent's own estimate when it gives one. Nothing that depends on the future (final totals, success, feasibility) enters the feature matrix; we assert this in code rather than trusting ourselves to have left it out. No weights, no logits, no environment internals: only the usage accounting that provider APIs already return, which is what makes the guard a wrapper rather than a modification, and a verifier rather than a believer. It never has to trust the agent's own estimate, because it wraps that estimate in a guarantee that holds whether the estimate is any good or not. The predictor behind the band is gradient-boosted quantile regression [27], so the width varies with the input. We split calibration and test by rollout, never by prefix: two prefixes of one run share its final total, and splitting them apart would leak each test answer into calibration.

Recalibrating as the stream moves. Frozen split-CQR is valid while calibration and deployment are exchangeable. Swap the underlying model, or point the guard at a new task type, and the resemblance is gone; so is the guarantee. Our response is to let the deployment stream recalibrate the guard from the runs that have already finished, in three ways. ACI [11] nudges its working confidence level

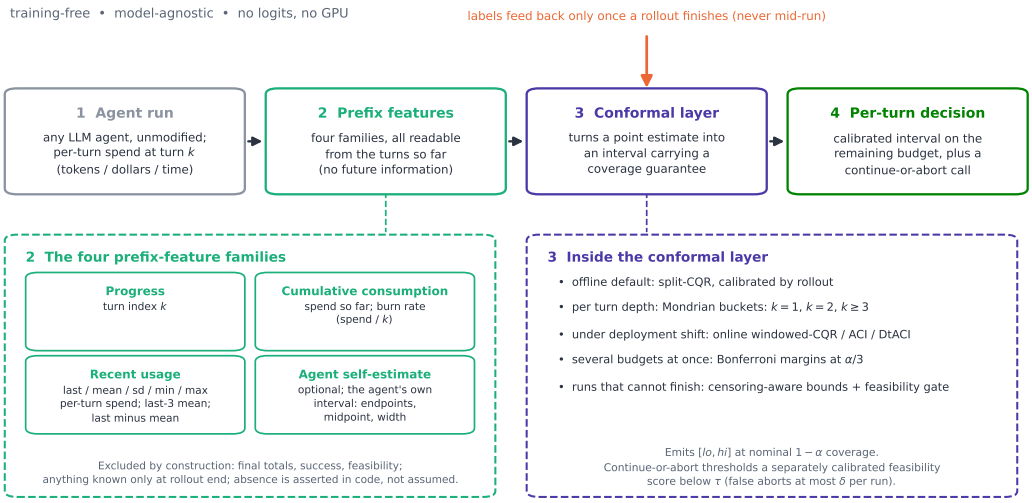


Figure 2: Budget Guard wraps an unmodified agent in an external calibration layer. Stages 1–4 share prefix features; stage 3 returns a calibrated remaining-budget interval (§§5.1–5.3), and a separately calibrated feasibility score drives continue-or-abort (§5.4). Warehouse abort economics use the feasibility gate alone and do not read the intervals. Labels become available only when a rollout finishes, so online recalibration never uses a still-running run.

up when it has been missing too often and down when it has been too safe. DtACI [12] runs several such nudges at different speeds and leans on whichever is currently working. Windowed-CQR keeps a sliding window of the 100 most recent scores and reads $\hat{q}$ straight off them; the idea is borrowed from SPCI [18], though we take the plain empirical quantile rather than fitting their model of the residual series, and the honest statement of what it provides is a coverage gap that grows with how far the window has drifted from the stream [19]. All three see a label only when its rollout ends, and hold their state fixed within a run. Two reporting rules follow. An interval with no upper bound covers everything and tells you nothing, so the fraction of unbounded intervals sits beside every coverage number. And because one margin is set by the many easy late-turn prefixes, a good average can hide a hole at turn one, where an abort decision is worth the most; we therefore also calibrate a separate $\hat{q}$ per turn-depth bucket, $k{=}1$, $k{=}2$, $k{\geq}3$ (Mondrian [20]).

Several budgets, and runs that cannot finish. A Warehouse run can fail three ways, out of time, capacity, or money, and it survives only if all three estimates hold. Covering each 90% of the time does not cover all three 90% of the time, because the misses accumulate; calibrating each at miscoverage $\alpha/3$ (Bonferroni) restores the joint guarantee by the union bound at the price of wider margins, and is our primary scheme. A tighter alternative shares one threshold across resources by ranking their errors jointly [21], but that rank is chosen on the same errors it then thresholds, so it carries no guarantee; we report it as an observation, and let both move under a model swap from a rolling window of finished runs. When a run is doomed we never learn y, only that it exceeded what was left, and dropping such runs removes exactly the hard cases. We calibrate one-sided upper bounds five ways (censored scores as $+\infty$, always valid and vacuous once censoring exceeds α; the known lower bound substituted; censored runs dropped, the naive comparison; reweighting by censoring probability, IPCW [22, 23]; and charging a censored run only against the resource that stopped it), and a classifier on the same features can decline to issue an interval at all. That changes what we can promise: coverage among the prefixes the gate lets through, not across all prefixes. It is weaker than a marginal guarantee, but it is what the construction supports, so it is what we claim.

The gate. The same feasibility score decides when to quit: abort at the first turn where it drops below a threshold τ. Since a run is aborted the moment its lowest score crosses τ, we set τ from the lowest scores of the feasible calibration runs, taking their δ-quantile with the finite-sample correction above, which bounds the probability of aborting a run that would have succeeded by δ, a per-run guarantee in the sense of conformal risk control [26]. Aborting a doomed run recovers what it had left to spend; aborting a good one throws away a success. Both land in the same currency, so the

Table 1: The four evaluation environments; only Warehouse binds several resources at once, and only Warehouse censors labels. Token budgets are the BAGEN context budgets in tokens; Warehouse budgets are set per task. In Warehouse, half of the prefixes (640 of 1,280 per model) belong to runs that can no longer finish and drive the censored-calibration evaluation.

Environment	Task	Budgeted resource(s)	Budget	# Models
Sokoban	box-pushing puzzle	tokens	2,500 tokens	7
Search-R1	multi-turn retrieval QA	tokens	3,500 tokens	7
SWE-bench	repository-level bug fixing	tokens	6,772–21,711 (per run)	5
Warehouse	multi-week management sim.	time, capacity, cost	per task, joint	5

gate is judged on Cost-of-Pass [25], total spend per successful run, rather than on coverage. The gate reads the feasibility score and nothing else; a variant that also aborts when spend so far plus the interval's calibrated lower bound exceeds the budget is evaluated on the token environments (Table 3).

4 Experimental setup

Data and environments. All experiments use logged rollouts from BAGEN [13] at a pinned dataset revision; Table 1 summarises the four environments. Sokoban, Search-R1, and SWE-bench are single-resource token tasks. Sokoban and Search-R1 use uniform context budgets of 2,500 and 3,500 tokens; on SWE-bench the official budget varies by run (6,772, 7,060, 8,265, 10,472, and 21,711 tokens across the five models) and we read each run's value from its own record. The label is the number of tokens still needed to complete the task under BAGEN's history-excluded per-turn token accounting. Warehouse is a multi-week management simulation with three jointly binding budgets (time in weeks, capacity in item-weeks, cost in simulated USD), with per-prefix ground truth for all three from the official records; half of its prefixes (640 of 1,280 per model) belong to runs that can no longer finish, which makes it our censoring test.

Protocol, baselines, and models. Rollouts are split 50/25/25 into train, calibration, and test, always by whole rollout. Every primary experiment is repeated over five random splits (seeds 0–4) and reported as the mean; the 19-run breadth count is one split per run, and we say so wherever it appears. Online methods consume the test stream in order with labels revealed only at rollout boundaries. We evaluate three shift scenarios: cross-environment (calibrate on Sokoban; deploy on Search-R1 and SWE-bench), model swap (calibrate on Claude Opus 4.7; deploy on four other agents' SWE-bench streams), and an external-resource model swap on Warehouse. Seven comparators run on the same splits and seeds as the guard: the agent's own interval; a feature-free constant-quantile interval, which measures what the learned component adds; split conformal on absolute residuals; the agent's own midpoint conformalized; the guard with the agent's estimate added to its features; frozen split-CQR under shift; and hard caps for stopping. Coverage is always reported against the 0.90 nominal level and beside the fraction of unbounded intervals. We evaluate seven agents (Claude Opus 4.7/4.6, Claude Sonnet 4.6, GPT-5.2-instant, GPT-5.2-Codex, Gemini 3.1 Pro, and Qwen3-235B), which crossed with environments gives 19 token-budget model$\times$environment runs (7 Sokoban, 7 Search-R1, 5 SWE-bench) of 52–128 rollouts and 85–3,715 prefix samples each. Appendix C gives the full protocol, baseline definitions, metric formulas, and implementation details.

5 Results

5.1 Within-run interval calibration

Figure 3 is the in-distribution picture on Claude Opus 4.7. The agent's own intervals cover 0.51, 0.23, and 0.35 of prefixes on Sokoban, Search-R1, and SWE-bench against a 0.90 target; offline split-CQR reaches 0.92, 0.90, and 0.96, and online windowed-CQR 0.92, 0.92, and 0.87, with no unbounded intervals anywhere. That split-CQR lands near 0.90 when calibration and test are exchangeable is what the construction guarantees; we report it as a check, not a result, and the findings are everything the guarantee does not cover. The intervals are bounded and sharp: windowed-CQR is 22% narrower than the feature-free constant-quantile baseline on Sokoban and 55% narrower

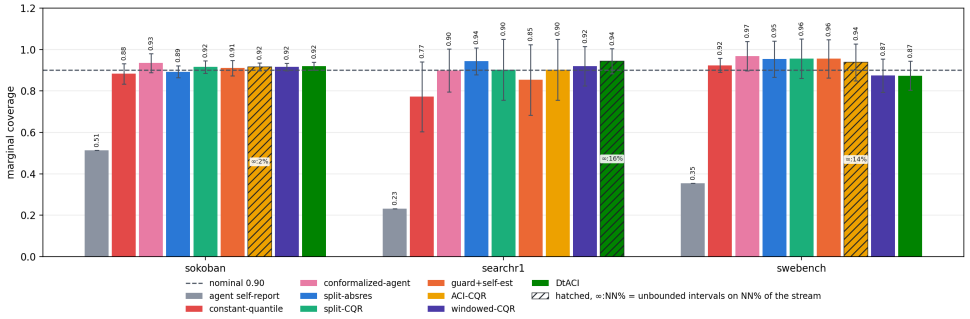


Figure 3: Conformal calibration closes almost all of the gap between agent self-reports and the 0.90 target, and windowed-CQR does so while maintaining bounded intervals throughout. Marginal coverage of the true remaining budget on Sokoban, Search-R1, and SWE-bench (Claude Opus 4.7 rollouts), α=0.10, averaged over five rollout-level split seeds; error bars are the standard deviation across those seeds. Hatched bars emit unbounded intervals on the annotated fraction of the stream, which raises coverage at the cost of informativeness. Agent self-report is a single value read from the logged records rather than a seed mean. Coverage, normalised width, unbounded fraction, and test-set sizes for all methods are in Table 5.

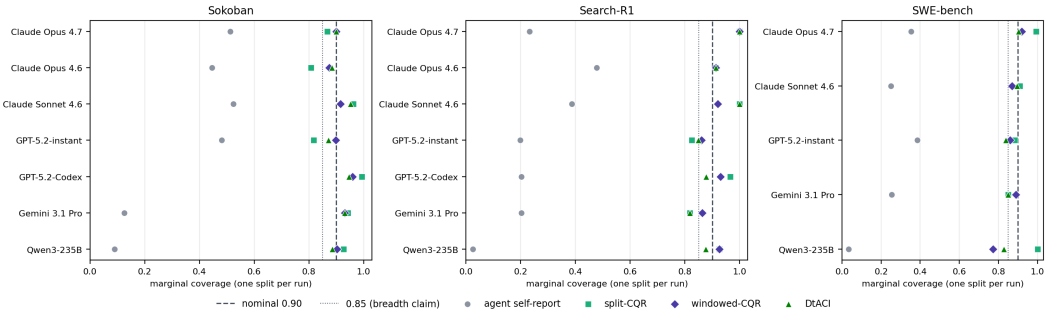


Figure 4: Every token-budget run, one split each (seed 0), no averaging: coverage of the agent's own interval and of split-CQR, windowed-CQR, and DtACI for each of the 19 model–environment runs, against the 0.90 target and the 0.85 line of the breadth claim. Windowed-CQR clears 0.85 on 18 of 19 runs and 0.90 on 11; the agents' own intervals clear neither anywhere. GPT-5.2-Codex on Sokoban carries no agent interval in its records. Table 7 lists the values.

on SWE-bench, though 23% wider on Search-R1, where constant-quantile is the sharpest bounded method (Table 5). ACI-CQR and DtACI sometimes post higher coverage, partly by emitting unbounded intervals on up to 16% of the stream. The agent's own estimate adds nothing as a feature: guard+self-est is 0.91 against split-CQR's 0.92 on Sokoban, identical on SWE-bench (0.96), and worse on Search-R1 (0.85 against 0.90). Figure 4 shows every run rather than one agent: across all 19 logged model–environment runs, windowed-CQR reaches at least 0.85 coverage in 18 of 19 and at least 0.90 in 11 of 19, against 14 and 8 for DtACI, on one split per run, and the agents' own intervals clear neither on any run (Table 7). The run of Figure 1 is typical rather than selected: over the 30 test rollouts of that split, split-CQR covers more prefixes than the self-report on 22, ties on 6, and trails on 2 (windowed-CQR: 24, 5, and 1).

5.2 Robustness under deployment shift

This is where the guarantee stops and the experiment starts. Figure 5(a) takes a guard calibrated on Sokoban and points it, unchanged, at Search-R1 and SWE-bench: frozen split-CQR falls to 0.51 and 0.10. Rollout-safe windowed-CQR, recalibrating only from runs that have finished, recovers 0.71 and 0.80 with no unbounded intervals. ACI and DtACI post higher numbers but need unbounded intervals on 60–88% of predictions to get there, which is recovery by surrender rather than by adaptation. Figure 5(b) swaps the agent instead of the task: a guard calibrated on Claude Opus 4.7 is deployed on Sonnet, Gemini, GPT, and Qwen SWE-bench streams. Windowed-CQR reaches 0.66–0.89, below target everywhere, but its seed-to-seed standard deviation is an order of magnitude smaller than frozen calibration's (0.006–0.011 against 0.122–0.224), so its failures are at least

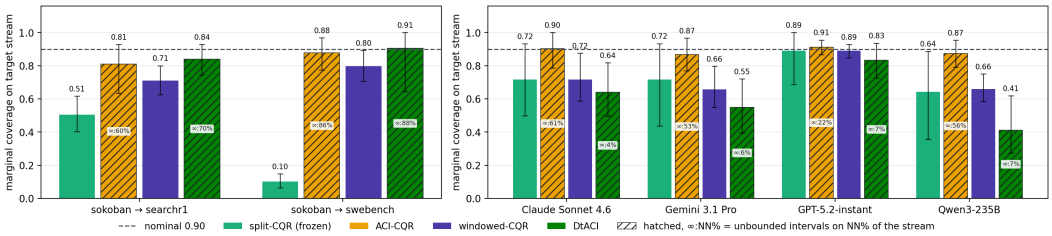


Figure 5: An environment change collapses frozen calibration and online recalibration recovers much of it; a model swap leaves every method below nominal coverage. Marginal coverage on the target stream. (a) A guard calibrated on Sokoban and deployed unchanged on Search-R1 and SWE-bench: frozen split-CQR falls to 0.51 and 0.10, and windowed-CQR recovers to 0.71 and 0.80. (b) A guard calibrated on Claude Opus 4.7 SWE-bench rollouts and deployed on four other agents: here frozen and online calibration are comparable on average (0.74 against 0.73) and neither reaches the target, though windowed-CQR varies far less across seeds. Hatched bars emit unbounded intervals on the annotated fraction of the stream (60–88% for the ACI-based methods in (a), and 22–61% (ACI-CQR) and 4–7% (DtACI) in (b)).

predictable. Online adaptation helps under shift and does not solve it: no bounded-interval method recovers the nominal level under a model swap, and any coverage number has to be read next to how many of its intervals were infinite.

5.3 Conditional and multi-resource calibration

A marginal number can hide a hole in the slice that matters. On Sokoban, split-CQR reaches 0.92 marginal coverage but only 0.86 at the first turn, exactly where an abort decision has the most leverage; Mondrian calibration by turn depth raises first-turn coverage to 0.93, is neutral to slightly negative on Search-R1, and neutral to slightly positive on SWE-bench, where the first-turn gap is already small (Table 8). What it buys is conditional validity, not a uniform improvement.

Table 2: Independent margins under-cover the joint event, Bonferroni fixes it in distribution, and nothing with a guarantee survives a model swap. Warehouse coverage against a nominal 0.90, mean over five rollout-level split seeds; above 0.90, coverage is bought with wider margins. Per-resource columns and the in-distribution joint column are Claude Opus 4.7; the final column is the joint event averaged over four swap targets. Bold marks the highest joint coverage in each column. "–" marks a scheme not run in that setting.

	per-resource			joint	
Calibration scheme	time	capacity	cost	in-dist.	model swap
agent self-report	0.87	0.67	0.71	0.42	0.34
independent margins (α)	0.99	0.89	0.92	0.80	–
Bonferroni ($\alpha/3$)	1.00	0.98	0.97	**0.95**	0.71
empirical copula	0.99	0.94	0.96	0.89	0.62
adaptive copula (online)	–	–	–	–	**0.87**

Table 2 is the multi-resource case. In Warehouse the agent's self-reports cover the joint event across time, capacity, and cost 0.42 of the time. Calibrating each resource independently at α lifts that only to 0.80: independent margins do not compose into joint coverage. Bonferroni at $\alpha/3$ restores 0.95, and the empirical copula reaches 0.89 with narrower margins but no guarantee. Under an external-resource model swap, frozen Bonferroni falls to 0.71, below its nominal level and below the 0.87 of the un-guaranteed adaptive copula, which recalibrates from finished rollouts. We report both and rest the claim on the one with the guarantee.

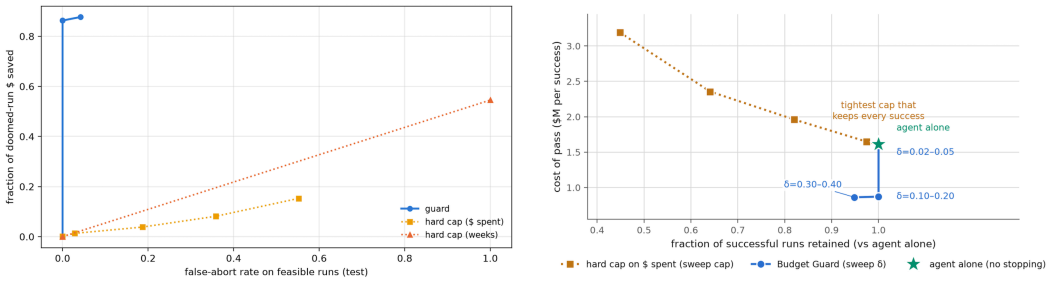


Figure 6: Warehouse economic impact from the feasibility gate alone, without remaining-budget intervals. At δ=0.10 the classifier-only rule avoids 86.3% of doomed-run spend with no observed false aborts, which no hard cap achieves at any setting. Simulated USD, mean over five seeds. (a) Every hard cap that saves more than 0.2% of doomed spend also stops runs that would have succeeded; the gate occupies the favourable region of the savings–false-abort trade-off. (b) Cost-of-Pass against the fraction of successful runs retained: the gate roughly halves cost per pass while retaining every observed success, whereas no cap improves on the unstopped baseline. Both panels are counterfactual replays of logged rollouts, not live deployments.

5.4 Economic impact

The gate is judged on money, not coverage. Figure 6 replays Warehouse with the classifier-only gate at δ=0.10: it avoids 86.3% of doomed-run spend with 0% observed false aborts. Hard spending caps cannot: the tightest dollar cap that preserves every successful run saves 0.2% of doomed spend, dollar caps reach meaningful savings only at false-abort rates of 19–55%, and the only week cap that saves anything (54.5%) aborts every feasible run. The gate cuts Cost-of-Pass from \$1.61M to \$0.87M, a reduction of 46%, while retaining every observed success; no cap improves on the unstopped baseline by more than a rounding error, and the tightest raises cost per pass to \$3.19M by discarding passes.

Table 3: Abort performance at δ=0.10 (mean over five seeds). Warehouse uses the classifier-only gate (native USD). Token abort is evaluated on Sokoban and SWE-bench only; both also evaluate a classifier+interval variant. Combining intervals reduces doomed savings at this operating point.

Environment	Rule	False abort	Doomed spend saved	Uses intervals?
Warehouse	feasibility only	0.0%	86.3%	No
Sokoban	feasibility only	5.8%	7.1%	No
Sokoban	feasibility+interval	5.8%	1.9%	Yes
SWE-bench	feasibility only	0.0%	3.1%	No
SWE-bench	feasibility+interval	0.0%	0.0%	Yes

Table 3 puts the same gate on the token tasks, and the picture changes. At the same δ=0.10, Sokoban and SWE-bench save only 7.1% and 3.1% of doomed token spend, with false-abort rates of 5.8% and 0%. Adding the interval trigger to the rule makes it more conservative at fixed δ (Sokoban savings fall to 1.9%, SWE-bench to 0%) and recovers none of the Warehouse-scale savings. This is the result behind keeping the two components apart. The intervals measure how much a run still needs, and §5.1 shows they measure it reliably; they are not a stopping signal, and used as one they only make the gate timid. What they are for is planning and monitoring: a deployer reads spend so far plus the upper bound against the budget to see whether a run still fits, as in Figure 1, or sizes the quota for a batch of runs before launching it. The gate is for stopping. The 86.3% is therefore a Warehouse result for the gate alone, not a general Budget Guard savings rate. Two further cautions: the dollars are simulated Warehouse dollars, not comparable to BAGEN's 28–64% token savings; and an observed 0% false-abort rate on a small test set is consistent with the δ=0.10 bound, not proof that the true rate is zero.

5.5 Censored calibration

Roughly half of Warehouse prefixes are censored: the run can no longer finish, so what it would have needed is never observed. Treating those runs as missing data is the wrong call. Dropping them yields only 0.50 coverage on the censored prefixes where cost is the binding resource, which are exactly the expensive failures. Treating censored scores as $+\infty$ is valid but vacuous at this censoring rate, and the informative treatments trade validity for sharpness in ways Table 10 makes explicit. The other lever is to abstain: feasibility-gated selective intervals hold coverage at 0.92–0.95 for capacity and cost, and 1.00 for time, on the prefixes they keep, at the cost of keeping between 51% and 89% of them (Table 9). That gate is swept over its own prefix-level tolerance δ_{gate}, a different quantity from the rollout-level δ of §5.4. One caveat governs these numbers: the recovered labels used to score censored prefixes are exact only for time, and carry a week-proportional interpolation with 12–27% median relative error for capacity and cost, so those two are coverage against an approximate label rather than ground truth.

6 Discussion and limitations

The paper shows that an agent's own remaining-budget estimate can be replaced, from outside, by an interval that is right as often as promised in distribution and degrades visibly rather than silently under shift; that no bounded method reaches the target under a model swap; that joint coverage over several budgets needs a joint correction and loses its guarantee under swap; and that a separately calibrated gate, not the intervals, is what saves money, and only where doomed runs are expensive and identifiable early. It does not show a general savings rate: the 86.3% is Warehouse, gate-only, and falls to 7.1% and 3.1% on token tasks at the same δ (Table 3). The joint results rest on one multi-resource environment, which cannot support a claim that they generalise. The in-distribution coverage numbers should be read as a check on the construction; the evidence for the method is in the shift, sharpness, joint, censoring, and economic results.

The breadth count is a single split: on seed 0, windowed-CQR reaches at least 0.85 coverage in 18 of 19 runs. Search-R1 leaves about 24 test prefixes per seed, which is why its standard deviations reach 0.10–0.17. Coverage alone does not distinguish a useful interval from a trivially wide one, so width and unbounded fraction are reported throughout; Figure 1 also shows the price of calibration, intervals roughly twice as wide as the agent's, which costs nothing when the question is whether the run still fits and matters when a tight point estimate is needed. Censored capacity and cost results are measured against an interpolated label. Finally, savings and Cost-of-Pass are counterfactual replays: they assume adding the guard would not change agent behaviour and that the logged sample is representative.

Broader impact. The intended effect is to reduce wasted compute, energy, and money on agent runs that cannot succeed; the Warehouse gate at $\delta{=}0.10$ illustrates that potential in one environment (86.3% of doomed-run spend avoided in replay, without using remaining-budget intervals). The corresponding risk is that a stopping threshold set too aggressively terminates runs that would have finished; δ calibration exists to bound that risk at a user-chosen level, and the counterfactual caveat above means a deployer should not treat the reported savings as a guaranteed live return when choosing δ, nor assume token environments will see Warehouse-scale savings.

7 Conclusion

Budget awareness can be supplied from outside the agent. A training-free layer over the usage counts an API already reports turns them into remaining-budget intervals with 0.90–0.96 coverage offline and 0.87–0.92 online on Claude Opus 4.7 against a 0.90 target, and at least 0.85 in 18 of 19 model$\times$environment runs on one split each, where the agent's own intervals manage 0.23–0.51. That guarantee assumes exchangeability, and the experiments map what happens without it: online recalibration recovers much of an environment shift, no bounded method reaches nominal under a model swap, and joint calibration works in distribution and not under swap. A separately calibrated feasibility gate avoids 86.3% of doomed-run spend in Warehouse replay at $\delta{=}0.10$, and far less on token tasks. Budget awareness is a measurement and a decision, and it works when the two are calibrated, evaluated, and claimed separately.

References

[1] Thomas Kwa et al. Measuring AI ability to complete long software tasks. In *NeurIPS*, 2025. arXiv:2503.14499.

[2] Yuxi Chen et al. Token economics for LLM agents: A dual-view study from computing and economics. arXiv:2605.09104, 2026.

[3] Longju Bai et al. How do AI agents spend your money? Analyzing and predicting token consumption in agentic coding tasks. arXiv:2604.22750, 2026.

[4] Yang Sui et al. Stop overthinking: A survey on efficient reasoning for large language models. *TMLR*, 2025. arXiv:2503.16419.

[5] Rui Wang et al. Harnessing the reasoning economy: A survey of efficient reasoning for large language models. arXiv:2503.24377, 2025.

[6] Linan Yue et al. Don't overthink it: A survey of efficient R1-style large reasoning models. arXiv:2508.02120, 2025.

[7] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *TMLR*, 2024. arXiv:2305.05176.

[8] Michael J. Zellinger, Rex Liu, and Matt Thomson. Cost-saving LLM cascades with early abstention. arXiv:2502.09054, 2025.

[9] Hen Davidov et al. Knowing when to quit: A principled framework for dynamic abstention in LLM reasoning. In *ICML*, 2026. arXiv:2604.18419.

[10] Yaniv Romano, Evan Patterson, and Emmanuel J. Candès. Conformalized quantile regression. In *NeurIPS*, 2019. arXiv:1905.03222.

[11] Isaac Gibbs and Emmanuel J. Candès. Adaptive conformal inference under distribution shift. In *NeurIPS*, 2021. arXiv:2106.00170.

[12] Isaac Gibbs and Emmanuel J. Candès. Conformal inference for online prediction with arbitrary distribution shifts. *JMLR*, 25(162):1–36, 2024. arXiv:2208.08401.

[13] Yuxiang Lin et al. BAGEN: Are LLM agents budget-aware? arXiv:2606.00198, 2026.

[14] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023. arXiv:2107.07511.

[15] Jing Lei, Max G'Sell, Alessandro Rinaldo, Ryan J. Tibshirani, and Larry Wasserman. Distribution-free predictive inference for regression. *JASA*, 113(523):1094–1111, 2018. arXiv:1604.04173.

[16] Kai Ruan et al. Doomed from the start: Early abort of LLM agent episodes via a recall-controlled probe cascade. arXiv:2607.06503, 2026.

[17] Xiaofan Zhou, Huy Nguyen, Bo Yu, Chenxi Liu, and Lu Cheng. Adaptive stopping for multi-turn LLM reasoning. arXiv:2604.01413, 2026.

[18] Chen Xu and Yao Xie. Sequential predictive conformal inference for time series. In *ICML*, 2023. arXiv:2212.03463.

[19] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. Conformal prediction beyond exchangeability. *The Annals of Statistics*, 51(2):816–845, 2023. arXiv:2202.13415.

[20] Vladimir Vovk. Conditional validity of inductive conformal predictors. In *ACML*, 2012. arXiv:1209.2673.

[21] Soundouss Messoudi, Sébastien Destercke, and Sylvain Rousseau. Copula-based conformal prediction for multi-target regression. *Pattern Recognition*, 120:108101, 2021. arXiv:2101.12002.

[22] Yu Gui, Rohan Hore, Zhimei Ren, and Rina Foygel Barber. Conformalized survival analysis with adaptive cut-offs. *Biometrika*, 111(2):459–477, 2024. arXiv:2211.01227.

[23] James M. Robins and Andrea Rotnitzky. Recovery of information and adjustment for dependent censoring using surrogate markers. In *AIDS Epidemiology*, pages 297–331. Birkhäuser, 1992.

[24] E. L. Kaplan and Paul Meier. Nonparametric estimation from incomplete observations. *JASA*, 53(282):457–481, 1958.

[25] Mehmet Hamza Erol, Batu El, Mirac Suzgun, Mert Yuksekgonul, and James Zou. Cost-of-pass: An economic framework for evaluating language models. In *ICLR*, 2026. arXiv:2504.13359.

[26] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *ICLR*, 2024. arXiv:2208.02814.

[27] Guolin Ke et al. LightGBM: A highly efficient gradient boosting decision tree. In *NeurIPS*, 2017.

A Background and guarantees

This appendix defines the concepts needed to interpret the results.

- **Coverage.** The fraction of cases in which the true remaining budget falls inside the predicted interval. Coverage of 0.90 means the true value is contained in about 90% of cases.
- **Calibration set.** A held-out set of completed trajectories used to set interval widths and stopping thresholds.
- **Rollout (run).** One complete agent attempt at a task.
- **Turn (k).** One agent step within a rollout.
- **Prefix.** The partial trajectory containing all turns completed through turn k.
- $\hat{q}$**.** The conformal threshold: a learned error margin added to the model's predictions to expand them into an interval. Throughout, $\hat{q}$ is the conformal threshold and $\hat{f}_{\mathrm{lo}}$, $\hat{f}_{\mathrm{hi}}$ are the quantile regressor's own predictions.
- α**.** The allowed miscoverage rate; the target coverage is $1-\alpha$. We use $\alpha{=}0.10$ throughout. A smaller α raises the target and generally widens intervals.
- **Width.** The size of the predicted range. Narrower is more informative at equal coverage: $[100, 115]$ is more useful than $[100, 300]$. An interval extending to infinity attains coverage while saying nothing.
- δ**.** The false-abort tolerance for the stopping rule. A larger δ permits greater savings on doomed runs while accepting more risk of aborting successful ones.
- τ**.** The abort threshold. The guard aborts at the first turn whose feasibility score falls below τ. It is not chosen by hand: it is read off the calibration rollouts as the value holding the false-abort rate at or below δ, so a smaller δ produces a more cautious τ.
- **Offline.** Calibrates once on completed runs and stays fixed during deployment.
- **Online.** Updates calibration after each completed run, using its newly observed remaining-budget labels.

Table 4 states, component by component, what is guaranteed and under what assumption. Four rows carry no finite-sample guarantee at all (windowed-CQR, the empirical copula, the censoring-aware bounds other than $+\infty$, and the marginal rather than selective reading of the gated intervals), and every claim resting on those is reported in this paper as an empirical observation.

B Per-run breadth results

Table 7 lists every model×environment run behind the breadth claims in §5.1; the "18 of 19" and "11 of 19" counts can be read straight off the windowed-CQR column. Table 8 gives conditional coverage by turn depth, Table 9 the feasibility-gate sweep, and Table 10 the censored upper-bound calibrations. Every number is read from the generated result tables, which one command regenerates and two checkers verify. Table 6 gives the numbers behind Figure 1.

Table 4: What each component guarantees, and what it does not. Statements we do not have are listed as such, so a reader can tell a theorem from an observation.

Method	Role in Budget Guard	Guarantee and scope
split-CQR	CQR intervals conformalized on a by-rollout calibration split; our offline default [10].	Marginal coverage $\geq 1-\alpha$ over a random (rollout, prefix) pair, under exchangeable rollouts. Pooling a rollout's prefixes is an empirical evaluation, not the guaranteed object [15].
Mondrian-CQR	split-CQR calibrated separately per turn-depth bucket [20].	Coverage $\geq 1-\alpha$ within each bucket, under exchangeability within the bucket; thin buckets fall back to the global quantile.
ACI / DtACI	Adaptive coverage levels recalibrated from the stream's finished rollouts [11, 12].	Long-run average coverage approaches $1-\alpha$. May emit unbounded intervals; we clamp the working level at 0.999 rather than emit empty sets, a conservative deviation.
windowed-CQR	Conformal quantile over a sliding window of the 100 most recent scores; an SPCI-inspired simplification [18].	No finite-sample guarantee; coverage gap bounded by window-to-stream drift [19]. Evaluated empirically.
Bonferroni joint	Calibrates each resource at $\alpha/3$ so all hold at once; our primary multi-resource calibration scheme.	Joint coverage $\geq 1-\alpha$ by the union bound; no data-driven selection; conservative, so realised coverage exceeds the target in distribution.
empirical copula	Shared copula rank across resources [21]; an adaptive variant updates the rank per finished rollout.	Empirical only: the rank is selected on the same scores it thresholds. Reported alongside Bonferroni, not as the guaranteed claim.
censoring-aware upper bounds	One-sided bounds for doomed runs [22–24].	One-sided upper coverage on the censored subset. $+\infty$ always valid but vacuous once the censored fraction exceeds α; the others empirical or assumption-dependent.
feasibility-gated intervals	A classifier on the same prefix features issues intervals only on predicted-feasible prefixes.	Coverage $\geq 1-\alpha$ conditional on passing the gate: selective, not marginal. Marginal coverage reported empirically.
stopping rule (τ)	Abort at the first turn whose feasibility score drops below τ [26]; scored by Cost-of-Pass [25].	False-abort probability $\leq \delta$ per run, from rollout-level calibration on feasible runs.

C Protocol, baselines, metrics, and implementation

This appendix records what §4 summarises: how the data is split, what each baseline is, how each reported number is computed, and the settings under which all of it runs. Every value below is fixed in the released code, and the parenthetical names are the modules that implement it.

C.1 Splits and seeds

Every split is by rollout, never by prefix (`split.group_split`). For a given seed the unique rollout indices are permuted with `numpy.random.default_rng(seed)` and allocated 50/25/25 to train, calibration and test; every prefix follows its rollout. Prefixes of one rollout share that rollout's final total, so splitting them apart would place each test rollout's answer into calibration. A split that is requested but comes out empty raises an error rather than degrading quietly: with an empty calibration set the conformal quantile of zero scores is $+\infty$, every interval becomes $[0, \infty)$, and coverage reads a meaningless 1.00 with no other symptom. Reported numbers are means over five seeds, 0 through 4.

C.2 Rollout-safe online evaluation

Online methods consume the test rows in their natural trajectory order, rollout by rollout and turn by turn. A prefix's label is the resource the run still needs to finish, which cannot be observed until the rollout ends, so a rollout's labels enter the update state only at its boundary and the state is frozen within a rollout (`conformal.aci_cqr`, `windowed_cqr`, `dtaci`, each taking a `groups` argument). The code rejects test streams whose rollout ids are not contiguous blocks: an id that reappears after a boundary would silently absorb a still-running rollout's labels and inflate coverage with no visible

Table 5: Full within-run results: every conformal method lands within 0.05 of the 0.90 target except the feature-free constant-quantile baseline on Search-R1, where the agent's own intervals miss by 0.39–0.67; windowed-CQR gives the narrowest bounded intervals on Sokoban and the second-narrowest on SWE-bench, while constant-quantile is sharpest on Search-R1. Claude Opus 4.7, nominal 0.90, mean$\pm$ standard deviation over five rollout-level split seeds. Normalised width is interval width divided by the central 80% range of the labels (lower is sharper). "Unb." is the percentage of predictions with an infinite upper endpoint; width is shown as "–" where at least one seed produced a majority of unbounded intervals. n_{test} is test prefixes per seed.

Env	Method	Coverage	Norm. width	Unb. (%)	n_{test}
Sokoban	agent self-report	0.51	–	–	–
	constant-quantile	0.88 ± 0.05	1.10	0	87
	split-absres	0.89 ± 0.03	1.12	0	87
	split-CQR	0.92 ± 0.03	0.88	0	87
	conformalized-agent	0.93 ± 0.05	1.44	0	87
	guard+self-est	0.91 ± 0.04	0.91	0	87
	ACI-CQR	0.92 ± 0.02	0.88	2	87
	windowed-CQR	0.92 ± 0.02	0.86	0	87
	DtACI	0.92 ± 0.02	0.88	0	87
Search-R1	agent self-report	0.23	–	–	–
	constant-quantile	0.77 ± 0.17	0.90	0	24
	split-absres	0.94 ± 0.06	1.60	0	24
	split-CQR	0.90 ± 0.15	1.14	0	24
	conformalized-agent	0.90 ± 0.10	1.96	0	24
	guard+self-est	0.85 ± 0.17	1.07	0	24
	ACI-CQR	0.90 ± 0.15	1.14	0	24
	windowed-CQR	0.92 ± 0.10	1.10	0	24
	DtACI	0.94 ± 0.06	–	16	24
SWE-bench	agent self-report	0.35	–	–	–
	constant-quantile	0.92 ± 0.03	2.87	0	161
	split-absres	0.95 ± 0.09	2.56	0	161
	split-CQR	0.96 ± 0.10	2.11	0	161
	conformalized-agent	0.97 ± 0.07	2.23	0	161
	guard+self-est	0.96 ± 0.09	1.97	0	161
	ACI-CQR	0.94 ± 0.09	–	14	161
	windowed-CQR	0.87 ± 0.08	1.29	0	161
	DtACI	0.87 ± 0.07	1.22	0	161

Table 6: The first held-out Sokoban rollout in the seed-0 test stream, turn by turn (Claude Opus 4.7, 2,500-token budget); the run plotted in Figure 1. $\surd$: the interval contains the true remaining spend; $\times$: it misses. "Spent + upper" is spend so far plus the guard's upper bound; "Gate" is the feasibility score and decision at δ=0.10 (abort below 0.02).

Turn	Spent	True rem.	Agent self-report	Guard interval (split-CQR)	Spent + upper	Gate
1	557	1,090	[1,000, 1,800] $\surd$	[460, 1,582] $\surd$	2,139 $\leq$ 2,500	0.99 continue
2	1,076	571	[600, 1,200] $\times$	[210, 1,102] $\surd$	2,178 $\leq$ 2,500	0.95 continue
3	1,380	267	[400, 900] $\times$	[162, 902] $\surd$	2,282 $\leq$ 2,500	0.99 continue

symptom. Revealing labels per prefix instead would use information a deployed guard does not have, so we report only the rollout-safe protocol.

C.3 Base predictor and features

The point and quantile predictors are LightGBM models [27] with `n_estimators`=200, `learning_rate`=0.05, `num_leaves`=15 and `min_child_samples`=10. CQR fits two quantile regressors, at $\alpha/2$ and $1-\alpha/2$; the absolute-residual baseline and the interval-driven stopping variant use a squared-error regressor; the feasibility gate is an LGBM classifier with the same settings. No hyperparameter search was run: these are the defaults fixed at the start of the project and used unchanged for every experiment, baseline included.

Table 7: Breadth across all 19 token-budget runs: windowed-CQR clears 0.85 on 18 of them and 0.90 on 11, while the agents' own intervals clear neither anywhere. Marginal coverage against nominal 0.90 on a single rollout-level split (seed 0). Four decimals are shown on the three guard columns so the two counts above are directly checkable; at three decimals SWE-bench/Gemini DtACI rounds to 0.850 and appears to clear 0.85 when it does not. "–" marks a run whose records carry no agent interval. †Warehouse is listed for dataset composition only; its 0.424 is joint coverage across three resources (Table 2), not a token-budget marginal.

Environment	Model	Rollouts	Prefixes	Agent	split-CQR	windowed-CQR	DtACI
Sokoban	Claude Opus 4.6	116	410	0.446	0.8077	0.8750	0.8846
	Claude Opus 4.7	123	355	0.513	0.8667	0.9000	0.9000
	Claude Sonnet 4.6	126	400	0.524	0.9623	0.9151	0.9528
	Gemini 3.1 Pro	126	389	0.125	0.9425	0.9310	0.9310
	GPT-5.2-instant	128	661	0.481	0.8176	0.8986	0.8716
	GPT-5.2-Codex	128	592	–	0.9933	0.9600	0.9467
	Qwen3-235B	126	1,038	0.090	0.9268	0.9024	0.8862
Search-R1	Claude Opus 4.7	66	87	0.233	1.0000	1.0000	1.0000
	Claude Sonnet 4.6	64	102	0.386	1.0000	0.9200	1.0000
	Claude Opus 4.6	52	90	0.478	0.9130	0.9130	0.9130
	GPT-5.2-instant	125	324	0.198	0.8256	0.8605	0.8488
	GPT-5.2-Codex	126	262	0.202	0.9649	0.9298	0.8772
	Gemini 3.1 Pro	83	85	0.202	0.8182	0.8636	0.8182
	Qwen3-235B	124	1,079	0.025	0.9259	0.9259	0.8765
SWE-bench	Claude Opus 4.7	64	712	0.354	0.9926	0.9185	0.9037
	Claude Sonnet 4.6	64	1,258	0.251	0.9101	0.8705	0.8957
	Gemini 3.1 Pro	64	2,327	0.254	0.8497	0.8899	0.8497
	GPT-5.2-instant	64	1,313	0.385	0.8840	0.8589	0.8370
	Qwen3-235B	60	3,715	0.034	1.0000	0.7727	0.8271
Warehouse	Opus 4.7 + 4 swap targets	64 ea.	1,280 ea.	0.424†	–	–	–

Table 8: Marginal coverage hides a first-turn deficit on Sokoban, and Mondrian calibration is what removes it. Coverage by turn-depth bucket, nominal 0.90, mean over five seeds. Search-R1 rollouts are short enough that no $k{\geq}3$ bucket exists.

Environment	Method	Marginal	$k{=}1$	$k{=}2$	$k{\geq}3$
Sokoban	split-CQR	0.92	0.86	0.93	0.96
	Mondrian-CQR	0.94	0.93	0.96	0.94
Search-R1	split-CQR	0.90	0.91	0.89	–
	Mondrian-CQR	0.89	0.89	0.89	–
SWE-bench	split-CQR	0.96	0.99	0.99	0.95
	Mondrian-CQR	0.97	0.99	0.99	0.96

The feature matrix (`features.build_xy`) is: the turn index k; cumulative spend; burn rate (spend/k); the last, mean, standard deviation, minimum and maximum per-turn spend; the mean of the last three turns; and the difference between the last turn and the mean. When the agent emits its own interval, four columns are added (the two endpoints, the midpoint and the width), and a missing estimate is left as `NaN`, which LightGBM handles natively. Quantities known only at rollout end (final totals, success, feasibility) are listed in a leak set and asserted absent from the matrix at build time, so the exclusion is enforced rather than intended.

C.4 Conformal methods and their settings

Every method uses the finite-sample corrected quantile: with n calibration scores the level is $\lceil (n+1)(1{-}\alpha) \rceil / n$, read with higher interpolation, returning $+\infty$ when that level exceeds 1. All intervals are clipped below at zero, since a remaining budget cannot be negative. $\alpha{=}0.10$ throughout.

Table 9: The feasibility gate trades retained volume for coverage on what it keeps: abstaining on predicted-doomed prefixes holds 0.92–0.95 coverage on capacity, where marginal calibration reaches only 0.89. Warehouse, mean over five seeds. "Kept" is the fraction of prefixes the gate admits, "caught" the fraction of doomed prefixes it correctly abstains on, "false abort" the fraction of feasible prefixes it wrongly rejects, and "coverage" the interval coverage among admitted feasible prefixes. Time binds in only 30% of censored runs and the gate catches almost all of them, so its row is flat. δ here is the gate's prefix-level tolerance, not the rollout-level false-abort tolerance of §5.4.

Resource	δ	Kept (↑)	Caught (↑)	False abort (↓)	Coverage (↑)
time (weeks)	0.02	0.88	0.93	0.00	1.00
	0.05	0.88	0.93	0.00	1.00
	0.10	0.88	0.93	0.00	1.00
	0.20	0.88	0.93	0.00	1.00
	0.30	0.88	0.93	0.00	1.00
capacity (item-weeks)	0.02	0.89	0.31	0.04	0.95
	0.05	0.78	0.64	0.05	0.95
	0.10	0.70	0.74	0.07	0.95
	0.20	0.57	0.82	0.19	0.95
	0.30	0.51	0.85	0.23	0.94
cost (USD)	0.02	0.86	0.60	0.05	0.92
	0.05	0.81	0.69	0.10	0.92
	0.10	0.80	0.72	0.10	0.92
	0.20	0.74	0.75	0.21	0.93
	0.30	0.74	0.76	0.21	0.93

split-CQR uses the score $\max(\hat{f}_{\text{lo}} - y,\ y - \hat{f}_{\text{hi}})$ and one global $\hat{q}$; it is the offline default. Mondrian-CQR applies the same score with one $\hat{q}$ per turn-depth bucket, split at $k{=}1$, $k{=}2$ and $k{\geq}3$; a bucket holding fewer than 10 calibration scores falls back to the global $\hat{q}$.

ACI updates the working level by $\alpha_{t+1} = \alpha_t + \gamma(\alpha - \text{err}_t)$. γ is selected on calibration data only: the calibration rollouts are split in half, ACI is streamed over the second half under the same rollout-safe revelation used at test time, and the $\gamma \in \{0.005, 0.01, 0.02, 0.05\}$ whose coverage lands closest to $1{-}\alpha$ is kept. One deliberate deviation from canonical ACI: where the canonical algorithm emits the empty set once $\alpha_t \geq 1$, we clamp the quantile lookup at 0.999 and emit a finite interval. The deviation is conservative in both coverage and the error feedback, and we state it rather than diverge silently.

windowed-CQR keeps a rolling window of the 100 most recent conformity scores, warm-started from calibration, and reads the interval half-width off that window's empirical quantile, so the score distribution itself tracks drift rather than reweighting a frozen calibration set. SPCI [18] is the inspiration, not the implemented method.

DtACI aggregates ACI experts at $\gamma \in \{0.005, 0.01, 0.02, 0.05, 0.1, 0.2\}$ with learning rate $\eta = 2.0$ and uniform mixing $\sigma = 0.01$. After each revealed label, experts are scored by pinball loss against β_t, the largest miscoverage level whose quantile would still have covered the observed score, and the working level is the weight-averaged α_t.

C.5 Baselines

Seven comparators run on the same splits and seeds as the guard.

1. **Agent self-report.** BAGEN's own per-sample indicators of whether the agent's interval contained the truth, read from the official estimation records. No calibration is applied.
2. **Constant-quantile.** Feature-free: the empirical $[\alpha/2, 1{-}\alpha/2]$ interval of the calibration labels, identical at every test prefix.
3. **Split-absres.** A LightGBM point prediction plus or minus the calibrated absolute-residual quantile.
4. **Conformalized-agent.** The agent's own interval midpoint used as the point predictor and conformalized on $|y - \text{mid}|$.

Table 10: Dropping censored runs under-covers exactly the expensive failures, and every informative repair trades validity for sharpness. One-sided upper-bound coverage on Warehouse at nominal 0.90, split by whether the resource was the binding constraint. $+\infty$ is always valid and always vacuous; "drop" is the naive comparison. Kaplan–Meier is a diagnostic, not a calibration option. Coverage for capacity and cost is measured against an interpolated label (12–27% median relative error), not ground truth.

Resource	Treatment	Feasible (↑0.90)	Cens. binding	Cens. non-binding	Binding frac.
time (weeks)	drop (naive)	1.00	1.00	1.00	0.30
	$+\infty$	1.00	1.00	1.00	0.30
	substituted bound	1.00	1.00	1.00	0.30
	IPCW	0.96	0.96	0.96	0.30
	per-resource bound	0.90	0.90	0.90	0.30
	Kaplan–Meier (diagnostic)	1.00	1.00	1.00	0.30
capacity (item-weeks)	drop (naive)	0.94	0.77	0.98	0.34
	$+\infty$	1.00	1.00	1.00	0.34
	substituted bound	0.99	0.98	1.00	0.34
	IPCW	0.93	0.75	0.98	0.34
	per-resource bound	0.91	0.74	0.98	0.34
	Kaplan–Meier (diagnostic)	0.94	0.78	0.99	0.34
cost (USD)	drop (naive)	0.88	0.50	0.97	0.42
	$+\infty$	1.00	1.00	1.00	0.42
	substituted bound	1.00	0.99	1.00	0.42
	IPCW	0.88	0.50	0.96	0.42
	per-resource bound	0.86	0.46	0.94	0.42
	Kaplan–Meier (diagnostic)	0.92	0.58	0.98	0.42

5. **Guard+self-est.** split-CQR on the guard features augmented with the agent's estimate columns.
6. **Frozen split-CQR.** split-CQR calibrated on the source environment or model and applied unchanged to the target stream.
7. **Hard caps.** Abort once spend so far exceeds a fixed cap. Dollar caps are swept at the 20th, 35th, 50th, 65th and 80th percentiles of realised run totals, and week caps at $0.5\times$ the 25th, $0.75\times$ the 50th and $1.0\times$ the 75th percentile of the task's time budget.

Every baseline sees the same training split, the same calibration split, the same test stream and the same seeds as the guard. BAGEN's SFT+RL estimator is not included as a baseline: it is a fine-tuned agent rather than an external wrapper, and re-running it would require training the underlying LLM, which is exactly what this work avoids.

C.6 Metrics

Coverage is the fraction of test prefixes whose interval contains the true remaining requirement. Normalised width is the interval width divided by the central 80% range of the labels, $q_{0.9}-q_{0.1}$, which makes environments whose token scales differ by orders of magnitude comparable; the median width is used wherever a method can emit unbounded intervals, and the across-seed summary is reported as "–" where at least one seed's median is itself infinite. Unbounded fraction is the share of predictions with an infinite upper endpoint, and it appears beside every coverage number.

Confidence intervals come from a cluster bootstrap that resamples whole rollouts rather than prefixes, with $B = 500$ resamples; per-sample binomial intervals are too tight here, because the prefixes of one rollout are correlated. Across seeds, the per-seed bootstrap draws are pooled and the 5th and 95th percentiles of the pooled distribution are reported as a 90% interval; these are recorded in the released result tables rather than plotted. The $\pm$ shown in Table 5 and the error bars in Figure 3 are both the standard deviation across the five seeds.

For the stopping rule, the false-abort rate is the fraction of feasible test rollouts that were stopped, and the doomed-savings fraction is the spend avoided on rollouts that could not finish divided by the spend those rollouts would otherwise have incurred. Cost-of-Pass is total spend under a policy

divided by the number of runs that still pass [25]. All economic results in this paper are Warehouse, which is priced natively in simulated dollars; token-environment economics appear only in Table 3.

C.7 Multi-resource and censored calibration

Warehouse binds time in weeks, capacity in item-weeks and cost in simulated dollars at once. Independent calibration sets each margin at α; Bonferroni sets each at $\alpha/3$, which restores a joint guarantee of $1-\alpha$ by the union bound and is the primary multi-resource calibration scheme. The empirical-copula variant shares one rank across resources and is tighter, but that rank is selected on the same calibration scores it then thresholds, so it carries no guarantee and is reported as an observation only. Under a model swap, an adaptive variant recomputes both the margins and the shared rank from a rolling window of recent three-resource error vectors, updated only once a rollout has finished.

For censored runs the one-sided upper bound is calibrated five ways (`censored.upper_qhat`): drop; infinity; bound; ipcw; and resource_bound. A Kaplan–Meier estimator [24] is a diagnostic rather than a sixth option: it returns $+\infty$ exactly when the upper tail is entirely censored. Table 10 reports the five calibrations together with the Kaplan–Meier diagnostic. Censored-stratum coverage for capacity and cost is coverage against an approximate interpolated label (12–27% median relative error), not against ground truth.

The feasibility gate is an LGBM classifier on the same prefix features. It issues intervals only on predicted-feasible prefixes, so the resulting guarantee is selective. The gate is swept at its own prefix-level tolerance $\delta_{\text{gate}} \in \{0.02, 0.05, 0.10, 0.20, 0.30\}$, which is a different quantity from the rollout-level false-abort tolerance δ of §5.4; Table 9 reports the full sweep.

C.8 Stopping rule and economics

The reported rule aborts at the first turn whose feasibility score falls below τ. Because the rule aborts a rollout exactly when that rollout's minimum prefix score crosses τ, τ is calibrated at the rollout level: it is set to the score at rank $\lfloor (n+1)\delta \rfloor$ among the sorted minimum scores of the n feasible calibration rollouts, which bounds by δ the probability of aborting a run that would have finished. δ is swept over $\{0.02, 0.05, 0.10, 0.15, 0.20, 0.30, 0.40\}$ for the frontier; $\delta{=}0.10$ is the operating point quoted in §5.4 and the abstract.

An interval-driven variant, which aborts when spend so far plus the calibrated lower bound exceeds the budget, is implemented in the released code and calibrated the same way at rollout level. It is not the rule behind the reported Warehouse economics, and we do not claim the two are equivalent. The economic numbers are counterfactual replays, not live measurements: each completed run is truncated where the guard would have aborted it and the avoided spend is summed.

C.9 Data, environment, and reproduction

All experiments use logged rollouts from BAGEN [13], fetched from its public Hugging Face release at the pinned revision `58189576e54b675fdd0e1d6c1c9f189c2992732f`. Its dataset card states no licence, so we use the data as publicly released for research and credit the originators rather than asserting terms the release does not state. The extractor reproduces BAGEN's turn-excluding-history token accounting exactly, with 0 of 355 mismatches against the official per-sample ground truth on the reference run.

The whole pipeline runs inside a container whose base image is pinned by digest (`python:3.12-slim-bookworm`), with numpy 2.5.1, pandas 3.0.3, LightGBM 4.6.0, scikit-learn 1.9.0, scipy 1.18.0 and matplotlib 3.11.0 resolved from a hash-locked requirements file. LightGBM is MIT-licensed; numpy, pandas, scipy and scikit-learn are BSD-3-Clause; matplotlib uses a PSF-style licence. The guard is CPU-only: no GPU, no model weights, and no access to logits or environment internals. No LLM is trained, fine-tuned or queried by us at any stage. A single command regenerates every figure and table in the repository and then machine-verifies the paper against what it just produced, exiting non-zero if any reported claim no longer matches.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Every headline number in the abstract and Section 1 is produced by an experiment in this paper. The abstract distinguishes interval calibration from abort economics, states that the 86.3% savings and Cost-of-Pass figures are Warehouse-only and do not use remaining-budget intervals, and reports token-environment abort results at the same δ. Scope limits are stated throughout: Claude Opus 4.7 for within-run coverage, 0.95 joint coverage in-distribution only (0.71 under model swap), and counterfactual replay rather than live deployment.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 states that the joint multi-resource result rests entirely on one environment and does not survive a model swap, that the breadth counts come from a single split rather than a seed average, that marginal coverage can hide under-coverage in slices such as early turns, and that the economic figures are counterfactual replays rather than live measurements. Appendix C adds the assumption-level caveats and Table 4 marks which components carry no finite-sample guarantee.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.

- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper states and proves no new theoretical results. Every guarantee is imported from prior work and Table 4 records what is guaranteed for each component.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Appendix C specifies the split procedure, rollout-safe protocol, LightGBM settings, feature list, every conformal variant, all seven baselines, every metric, and the pinned data revision and software environment.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.

(b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

(c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: The data is fully open (public BAGEN rollouts at a pinned revision). The code is not attached to this submission; the full pipeline will be released under an MIT licence on acceptance, together with a digest-pinned Dockerfile and a single entry point that regenerates and verifies every reported number. Appendix C specifies everything needed for an independent reimplementation in the meantime.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4 and Table 1 give the environments and budgets; Appendix C gives the rest, including the exact sweep grids for hard-cap baselines, the feasibility gate, and δ.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Every headline number is a mean over five rollout-level split seeds. Figure 3 carries across-seed standard-deviation error bars; Table 5 reports the same quantity as a $\pm$ beside each coverage value. Appendix C specifies the cluster bootstrap ($B = 500$, resampling whole rollouts).

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix C states that the entire pipeline is CPU-only: no GPU, no model weights, and no LLM training or querying. All experiments fit gradient-boosted trees on tables of at most a few thousand rows inside one digest-pinned container.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: We have reviewed the NeurIPS Code of Ethics and believe this work conforms. The research uses only an existing, publicly released dataset of logged agent rollouts; it involves no human subjects and no new model release.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 6 contains an explicit broader-impact paragraph on wasted compute versus false-abort risk, with δ calibration as the mitigation.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: We release no model weights, no pretrained language model, and no scraped data. The artifact is CPU-only calibration code whose only effect is to stop an agent's own run earlier.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: BAGEN [13] is cited and pinned to an exact revision; its dataset card states no licence, so we assert none on its behalf and credit the originators. Appendix C lists the software stack with exact versions and licences (LightGBM MIT; numpy, pandas, scipy, scikit-learn BSD-3-Clause; matplotlib PSF-style).

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: No new asset is released with this submission. The implementation will be released on acceptance; no new dataset is created.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: All experiments are offline analyses of pre-existing logged agent rollouts.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human subjects were involved.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: No LLM is a component of the core method. We neither train, fine-tune nor query any LLM in this work.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.